The Hodgemix Ternary Transformer: An End-to-End Description of the Merged Architecture, its Construction, File Format and Execution (Version 3: Gap Selection, Window Lock, One Ricci–DeTurck Step and L-A Towers)

Gregory Disney-Leugers
Encryptid Systems

2 November 2026

Abstract

This is the third version of the end-to-end description of the hodgemix ternary transformer. It describes the code of record pulled fresh from the build machine on 2 November 2026 (`hodgemix-go`, `main.go` dated 2 November 2026, 03:05). `hodgemix` assimilates open-weight safetensors checkpoints into a ternary BitNet b1.58 base [5]. The output is a *Hodge* model: its `config.json` carries `model_type` `hodge` and `architectures` `HodgeForCausalLM`, the names read by the hkruntime runtime. The Champ WebGPU runtime and its BNZ4 container are *legacy*, but their layout pass, compatibility check, BNZ compression and round trip still run in every build.

Since version 2 the per-tensor core has changed in four places.

- The Re-Basin/Hungarian seed is replaced by *gap selection*: a source fibre is copied only when Lemma 4.1 of the lattice paper certifies it.
- The Kuramoto/KTE tick anneal of the band lock is replaced by a *window certificate* on the measured phase.
- Every synthesized tensor takes *one explicit Ricci–DeTurck step* of its residual against the base (package `train`), with lattice necks capped to the base.
- A tensor with no locked mode keeps its curl residual for that step. It is no longer reset to the base grid.

Multimodal input tensors of contributors are no longer written. The only towers in an output are the L-A vision and audio towers built on the base embedding.

The run also writes `volume.safetensors` and a new `vocab.float.safetensors`. Theorems 6.12–6.14 of the Multi-Radix Geometry paper [12] remain paper-only: no code computes a or q. We give the exact stage order with its log names, then what is implemented and what is only specified. Quality numbers are smoke tests, and none has been recorded with this code.

Contents

1 Introduction **2**

2 The architecture as a transformer **3**
2.1 Dataflow . . . 3
2.2 Tokenizer and union vocabulary . . . 3
2.3 Embedding and residual stream . . . 4
2.4 The block . . . 5
2.5 Gemma 4 inputs and per-layer widths . . . 6
2.6 Towers in the forward . . . 6

2.7 Mixture-of-experts blocks and gates . . . 7
2.8 BitLinear: weights as digit strings . . . 8
2.9 Head . . . 9
2.10 File format and loading . . . 9
2.11 Calibration set . . . 10

3 The assimilation pipeline **10**
3.1 Index, grid and alignment seed . . . 11
3.2 Smoothed commuting 1-form on the grid . . . 12
3.3 Hodge split and the window lock . . . 13
3.4 One Ricci–DeTurck step (package `train`) . . . 13
3.5 No band, no lock . . . 14
3.6 Anneal early stop: limb lock only . . . 14
3.7 Curl set: ternary TIES over all contributors . . . 14
3.8 Base residual, scale, pin-mean . . . 14
3.9 Ternarization, body limbs and neck limbs . . . 14
3.10 Limb stopping rules . . . 15
3.11 TOWERS stage . . . 15
3.12 OV chart . . . 16
3.13 Surgery cap . . . 16
3.14 Repair . . . 17
3.15 Champ check (legacy) . . . 17
3.16 Run file and preflight . . . 17

4 Weight volumes and the forward cochain **18**

5 Tools outside the pipeline **19**

6 Measured behaviour (smoke test) **19**

7 Implemented versus specified **19**

8 Open issues **21**

9 Changelog **22**

A Claim index (file: function) **24**

1 Introduction

BitNet b1.58 [4] constrains every linear layer of a transformer to weights in $\{-1, 0, +1\}$, with one scale per tensor and 8-bit activations. BitNet b1.58 2B4T [5] is the first open native 1.58-bit model at two billion parameters. Its architecture (Table 1) is a Llama-style decoder with:

- a squared-ReLU gated FFN [23];
- rotary embeddings [20] with $\theta = 5 \cdot 10^5$;
- grouped-query attention [22] and RMSNorm [21];
- extra sub-layer norms before the output projections, in the style of Sub-LN [24];
- a tied embedding and head.

`hodgemix` takes a ternary model of this block as the fixed base. It assimilates other open-weight models (Qwen, Llama 3, VaultGemma, Gemma 4) into it through:

- the Hodge–Kuramoto alignment of [1] and the DTFT selective lock of [2], as now modified (§3.1–§3.4);
- the OV chart, the surgery cap and repair.

Version 3 describes the pipeline as it now runs:

- the output is named for the Hodge runtime (§2.10);
- Gemma 4 language-model inputs are read, and their multimodal tensors are set aside (§2.5);
- vision and audio towers are the L-A towers only (§2.6, §3.11);
- the per-tensor core is the one listed in the abstract.

Scope. The description is of `hodgemix-go` as pulled from the build machine on 2 November 2026 (`main.go` 2 November, 03:05). Every statement about code was checked against that tree, and against nothing older. It was compared file by file with the 25 October copy that the first draft of this version had wrongly used (§9).

We built and tested the tree with Go 1.27.3. `go build` succeeds, and the packages `eval`, `itt`, `spm2json`, `tower`, `train` and `turn` pass. The root package does not complete:

- `TestMarriedTopologyRun` and `TestMarriedTopologySelfIsOnePermutation` end the test binary through `os.Exit`, at the new vocab-float step (`vocab: residual cell N holds no channel`).
- With those two skipped, the root package gives 52 passing, 8 skipped and 1 failing test. The failure is the stale `TestConfigFollowsTensors` (§8).

No model-scale run was made with this code. The newest run log in the tree (`runs/hodge-4-instruct.log`, 27 October) predates it.

Table 1: Shapes of the 2B4T base. The reference forward reads them from the base's `config.json` (`eval/arch.go`); they are not hard-coded. A Gemma 4 base may carry per-layer widths (§2.5).

Symbol	Value	Meaning
D	2560	residual width (`hidden_size`)
F	6912	FFN width (`intermediate_size`); per layer F_ℓ = `FBy`$[\ell]$ if set
L	30	layers
H, H_{kv}, d_h	20, 5, 128	query heads, KV heads, head dim; group size $G = H/H_{kv} = 4$
V	128 256	base vocabulary (the union vocabulary is larger, Table 2)
θ	500 000	RoPE base
ϵ	10^{-5}	RMSNorm epsilon

2 The architecture as a transformer

2.1 Dataflow

Figure 1 shows one token through the model. All matrix products inside a block are BitLinear (§2.8); all other parameters (norm gains, embedding) are BF16.

2.2 Tokenizer and union vocabulary

With `-vocab union` (default) the output carries its own `tokenizer.json`, `tokenizer_config.json` and `config.json` (`vocab.go: buildUnion`). Let $\mathcal{V}_b$ be the base vocabulary and $\mathcal{V}_s$ that of source s.

1. *Ids.* Every $t \in \mathcal{V}_b$ keeps its id. Each source token whose string is not in $\mathcal{V}_b$ is appended in the source's id order; then the source's added (special) tokens the base lacks.
2. *Merges.* The base's BPE merges, then each source merge whose result is in the union and that the base does not already have; pre-tokenizer, normalizer and decoder are the base's.
3. *Embeddings.* Base rows are copied. An appended token's row is its source embedding mapped into the base residual space by the least-squares map over the shared strings $\mathcal{S} = \mathcal{V}_b \cap \mathcal{V}_s$:

$$E_b[t] - \mu_b \;\approx\; (E_s[t] - \mu_s)\, M, \qquad M = \arg\min_M \sum_{t \in \mathcal{S}_{\text{fit}}} \|E_b[t] - \mu_b - (E_s[t] - \mu_s)M\|^2, \tag{1}$$

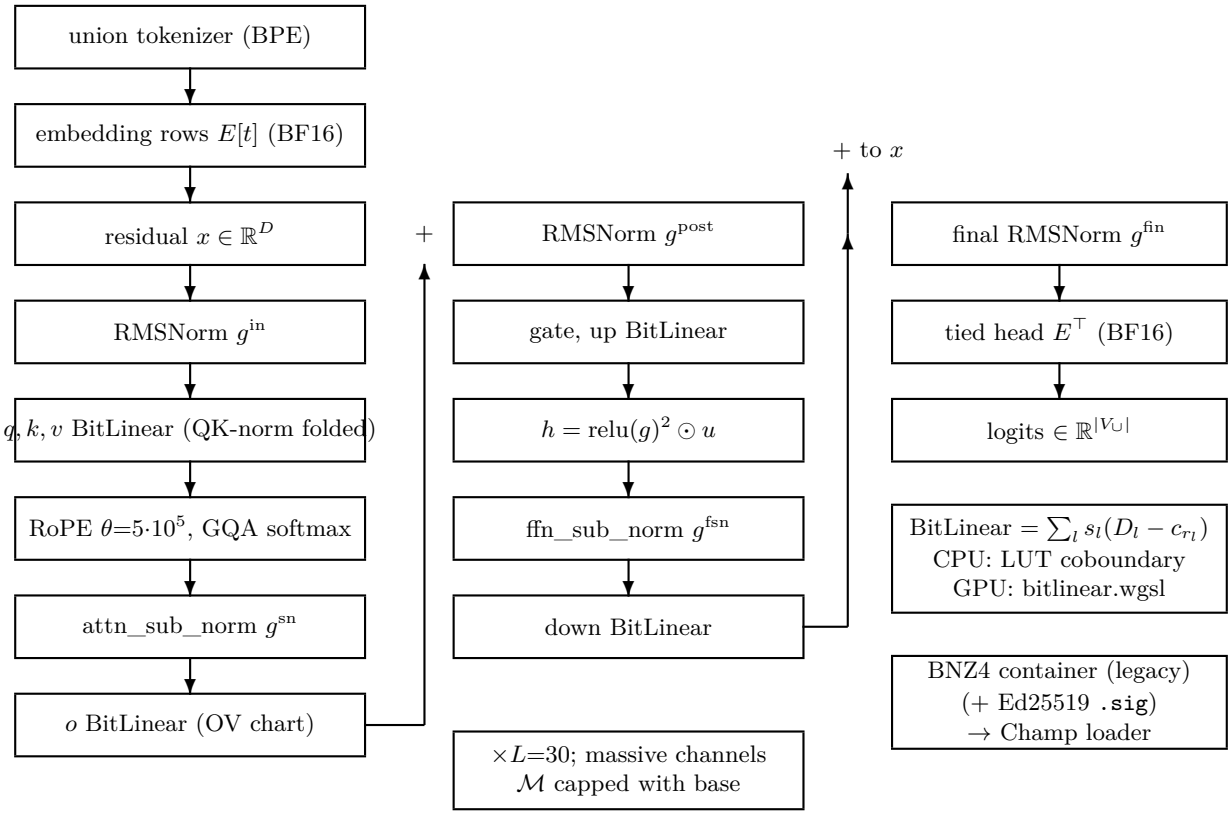


Figure 1: One token through the hodgemix transformer. Left: attention sub-layer; middle: FFN sub-layer, repeated for 30 layers with residual adds; right: head, BitLinear arithmetic and container. Every BitLinear input is quantized per token to int8 by absmax.

solved by Cholesky on the normal equations $X_s^\top X_s M = X_s^\top X_b$ (`vocab.go: residualMap, cholSolve`). $\mathcal{S}$ is shuffled with a fixed seed (1); 90% is used to fit, the held-out 10% gives R^2. The appended row is $\mu_b + (E_s[t] - \mu_s)M$.

Because the head is tied, the same rows serve as output embeddings. Table 2 gives the reported fits; the map explains a quarter to a third of the variance, so appended tokens enter with a partial embedding.

Table 2: Union vocabulary fits reported on 15 October.

Source	shared strings	held-out R^2	union size
Qwen3-0.6B	109 566	0.252	170 359 (42 077 + 26 added)
Llama-3.2-3B-Instruct	128 000	0.357	128 259 (3 added)

2.3 Embedding and residual stream

The embedding is a BF16 matrix $E \in \mathbb{R}^{|V_\cup| \times D}$; token t gives $x^{(0)} = E[t]$. In BNZ4 it is always written as row-range segments named `model.embed_tokens.weight[r0:r1]` with at most `EmbChunkRows`= 8192 rows of the tensor's own width per segment ($8192 \cdot D \cdot 2$ bytes; this was a fixed $8192 \cdot 2560 \cdot 2$ bytes before), because the loader schedules embeddings only by those names (`bnz4/container.go`). The residual stream $x \in \mathbb{R}^D$ is carried in float32 between sub-layers.

Massive channels. Following [11], a small set of residual coordinates carries values orders of magnitude above the median, sustained by a few super weights [19]. In `eval.FindSingular` the base's reference forward is run over the calibration set (§2.11); for each layer ℓ the energy $e_c^{(\ell)} = \sum_{\text{tokens}} (x_c^{\text{attn}})^2 + (x_c^{\text{mlp}})^2$ is accumulated over the residual after the attention add and after the FFN add, and channel c is massive if $\sqrt{e_c^{(\ell)}} \geq \rho \cdot \text{median}_{c'} \sqrt{e_{c'}^{(\ell)}}$ at some layer, with $\rho = 20$ (`-surgery-ratio`). FFN *spike units* at layer ℓ are those whose peak $\max_{\text{tokens}} |h_u|$ is $\geq \rho$ times the layer's median unit peak. On 2B4T this rule returns 17 channels at 41–60× the median, while the companion paper's trace identifies 9 channels [3]; §7 discusses the difference.

2.4 The block

For layer ℓ (dropping ℓ), with $\text{RMS}_g(x) = g \odot x / \sqrt{\overline{x^2} + \epsilon}$ and Q_8 the per-token absmax int8 quantizer $Q_8(y) = \big(\text{round}(127y/\max|y|),\ \max|y|/127\big)$ (`eval.go: quant`):

$$a = \text{RMS}_{g^{\text{in}}}(x), \qquad (q, k, v) = (W_q, W_k, W_v)\, Q_8(a), \tag{2}$$

$$\tilde{q}, \tilde{k} = \text{RoPE}_\theta(q), \text{RoPE}_\theta(k), \qquad o_h = \sum_{\tau \leq t} \text{softmax}_\tau\Big(\tfrac{\tilde{q}_h \cdot \tilde{k}_{g(h),\tau}}{\sqrt{d_h}}\Big) v_{g(h),\tau}, \tag{3}$$

$$x \leftarrow x + W_o\, Q_8\big(\text{RMS}_{g^{\text{sn}}}(o)\big), \tag{4}$$

$$f = \text{RMS}_{g^{\text{post}}}(x), \qquad h = \max(W_g Q_8(f), 0)^2 \odot W_u Q_8(f), \tag{5}$$

$$x \leftarrow x + W_d\, Q_8\big(\text{RMS}_{g^{\text{fsn}}}(h)\big), \tag{6}$$

with $g(h) = \lfloor h/G \rfloor$. RoPE rotates pairs $(d, d + d_h/2)$ at frequency θ^{-2d/d_h} (`eval.go: rope`). The final hidden state is $\text{RMS}_{g^{\text{fin}}}(x)$ and the logits are $E\,\text{RMS}_{g^{\text{fin}}}(x)$ (`eval.go: logits`).

Wedge gate in the reference forward (new since 25 October). The two residual adds above are no longer unconditional in `eval/eval.go: feed`. Before each add, `admitWrite` (`eval/wedge.go`) computes the oriented area of the face formed by the carry x and the sub-layer write y (attention output, then FFN output):

$$w = \sum_c \Sigma_c(x)\, \Sigma_c(y), \tag{7}$$

over the residual cells c of the volume layout, or channel by channel when there is none. The write is added only when $w > 0$. When $w \leq 0$ it is refused, the carry stays, and the edge is removed from the recorded cochain. The sign, area and decision are recorded per layer (`FlowLayer.Wedge`, `Sign`, `Keep`).

The gate has no flag. It is applied in every forward of the reference model, including CALIBRATE, the repair statistics, EVAL and generation, and it holds for stock models too. No eval number in this paper was measured with it; the smoke-test figures of §6 predate it. The code calls it "no paper equation".

Folded QK-norm. Qwen3 applies an RMSNorm with gain $\gamma^q \in \mathbb{R}^{d_h}$ to each query (and key) head; Champ has none. For every contributor whose config has QK-norm (`attention.go: prepQK`), each head h of W_q (resp. W_k) is replaced by

$$W'_{[hd_h+d,\,:]} = W_{[hd_h+d,\,:]} \cdot \gamma_d \cdot \sqrt{d_h \,/\, \|W_{[h]}\, \text{diag}(g^{\text{in}})\|_F^2}, \tag{8}$$

a static replacement of the per-token norm by the head's Frobenius scale under the input gain. The base is treated the same way; `q_norm`/`k_norm` are then dropped from the output (`attention.go: champLayout`). RoPE frequencies of a contributor with a different θ are remapped to the target θ (`ropeRemap`).

Model families. The reference forward (`eval/model.go`, `eval/arch.go`) and (per version 2) the Champ kernels run block families from the base's own configuration: `2b4t`, `llama`, `qwen`, `gemma` (including Vault-Gemma) and, in the reference forward only, `gpt-oss`. `eval/arch.go` now also maps `qwen2_vl`, `qwen2_5_vl`, `qwen2_5_vl_text`, `qwen3_vl` and `qwen3_vl_text` to the Qwen block, reads `rope_theta` from `rope_parameters` when `rope_scaling` is absent, and treats `mrope` as the default 1-D RoPE. Per family:

- gate activation $\phi \in \{\text{relu}^2,\ \text{silu},\ \text{gelu}_{\text{tanh}}\}$, selected in Champ by the override `ACT` of `norm.wgsl` (0, 1, 2), so $h = \phi(W_g f) \odot W_u f$;

- RMSNorm gain g or, for Gemma (`GEMMA`= 1), $(1 + g)$; an all-ones inserted sub-norm is kept the identity under either rule;
- embedding scale $\sqrt{D}$ for Gemma (`embed.wgsl SCALE`);
- attention denominator $\sqrt{d_h}$ or $\sqrt{\texttt{query_pre_attn_scalar}}$ (`attention.wgsl DENOM`);
- RoPE frequencies θ^{-2d/d_h}, divided by a factor (linear) or blended between the llama3 low/high frequency bands (`rope.wgsl SCALING, FACTOR, LOW_FF, HIGH_FF, ORIG`).

Champ's runtime (the `bitnet/` tree) is not part of `hodgemix-go`; the following is carried from version 2 and was not re-checked. Its `LoadConfig` refuses other activations, other RoPE scalings, attention bias, and any configuration in which `layer_types` or `use_sliding_window` turns on sliding-window attention. In `hodgemix-go`, `champ.go`: `champRuntime` refuses `model_type gpt_oss` or any name containing `moe` or `mixtral`, and now accepts the RoPE types `default`, `linear`, `llama3` and `mrope`, read from `rope_scaling` or `rope_parameters` (`eval.RopeDict`).

Sub-norms. If a base lacks `attn_sub_norm` or `ffn_sub_norm`, they are added at unit gain (`champLayout`), so that any base takes Champ's layout. An `lm_head.weight` byte-identical to the embedding is dropped.

OV chart. Head h writes $O_h V_g a$. In the output, $V_t = V_b$ exactly (base codes and scale), and $O_{t,h}$ carries the merged circuit to first order (§3.12). Attention therefore reads values in the base's coordinates.

2.5 Gemma 4 inputs and per-layer widths

A Gemma 4 checkpoint stores its text decoder under `model.language_model.*` and its multimodal weights beside it. `main.go`: `languageModel` detects that prefix. `embedKey` returns the token embedding as `model.embed_tokens.weight` or `model.language_model.embed_tokens.weight`, whichever the file has.

`asideTensor` keeps a tensor out of the text BitLinear mix *and out of the output*. It marks:

- tower weights (`audio_tower`, `vision_tower`, `embed_audio`, `embed_vision`; `towerTensor`);
- any further `embed_tokens`, the `per_layer_*` inputs, `layer_scalar` and `post_per_layer_input_norm`.

`bnzmix.*`, `model.embed_tokens.weight` and `lm_head.weight` are never aside. For a language-model checkpoint, every key outside `model.language_model.` is skipped as well.

Aside tensors are skipped in STAGE 1 (no MRH keys), in the topology pass and in preflight's role check. *They are not written.* After STAGE 3/6, `main.go` counts the aside tensors of the base and all sources and logs `TOWERS N encoder and per-layer tensors of the inputs are not written; the output's towers are the L-A towers`. The count goes into the report as `not_written`.

Two things are gone from the code: the earlier `carriedTensor`/`copyTowers` path that copied these tensors into the output, and the Gemma 4 encoder assimilation (`assimtowers.go`, deleted). The README records the consequence. `hodge-4.av` had carried 1 162 Gemma 4 tensors, while `hodge-4.la` holds no Gemma 4 tensor apart from 31 one-element values.

Gemma 4 layers are not uniform: some layers have a double-wide MLP, and full-attention layers have a wider head. The topology record (`topology.go`) therefore carries optional per-layer widths:

- `FBy`$[\ell]$, the intermediate size;
- `KVBy`$[\ell]$, equal to $H_{kv} d_h$;
- `QBy`$[\ell]$, equal to $H d_h$.

`fAt`(ℓ) returns F_ℓ, and `finishF` drops `FBy` when every layer has the model's F.

2.6 Towers in the forward

The reference forward (`eval/tower.go`, unchanged since 25 October) accepts tower features as layer-0 states. A vision image gives 196 patch tokens (16×16 patches of a 224×224 image); audio gives one token per 20 ms.

Each feature vector $f \in \mathbb{R}^{d_f}$ ($d_f = 46$ for vision, 240 for audio) is projected to the residual width by the stored projector (`tower.Project`):

$$x = \mu_{\text{row}} + P\,(f - \mu_f) \quad \text{rescaled to the base embedding row RMS}, \qquad P \in \mathbb{R}^{D \times d_f}. \tag{9}$$

The state is installed at reserved token ids one past the text embedding: vision from slot $|V|$, audio from slot $|V|+196$ (`model.install`, `Session.Vision`). At such an id the slot state replaces the embedding row (decision D3 as cited in the code).

Each slot also carries a phase ϕ_t, the argument of its patch- or band-mean readout. RoPE planes $p = 56, \ldots, 63$ of every head turn by $\text{pos} \cdot f_p + \phi_t$, and planes 0..55 turn by the standard angle. Text tokens have $\phi_t = 0$ (`eval.go`, D22). Before use, `checkTower` recomputes the operator's σ and fixed point $z^\star$, and refuses a file whose stored constants differ by more than $2 \cdot 10^{-5}$. The forward accepts `model_type hodge` as the ternary family (`ternaryFamily`).

`towerwasm/` builds the same `tower` package to WebAssembly for the page, so features in the browser are the ones the projectors were fitted to. The D-numbered decisions cited in code comments (D3–D22) are in `transfractal_tensor.tex`, which is not in the `hodgemix-go` tree. `DECISIONS.md` holds only decisions 1–11 and C1–C10.

2.7 Mixture-of-experts blocks and gates

Three mechanisms of mixture-of-experts type exist in the code. None is part of a Champ-loadable hodgemix output.

(G1) gpt-oss top-k router (reference forward). For a gpt-oss checkpoint (`num_local_experts`, `num_experts_per_tok`), the FFN of layer ℓ is (`eval/eval.go: mixture, topkSoftmax, swiglu`)

$$z = W_r f + b_r, \qquad I = \text{Top}_k(z) \text{ (lower index wins ties)}, \qquad \pi_e = \frac{e^{z_e}}{\sum_{e' \in I} e^{z_{e'}}},\ e \in I, \tag{10}$$

$$h_e = \big(\text{clip}(u_e, -\beta, \beta) + 1\big)\, \bar{g}_e\, \sigma(\alpha \bar{g}_e), \quad \bar{g}_e = \min(g_e, \beta), \qquad y = \sum_{e \in I} \pi_e\, W_d^{(e)} h_e, \tag{11}$$

with g_e, u_e the de-interleaved gate/up outputs of `experts.gate_up_proj` (even index gate), α the SwiGLU slope and β the limit (`swiglu_limit` 7 in the released configurations [15]), biases included. Experts stored as MXFP4 are evaluated as a 16-entry E2M1 table gather times an E8M0 block scale 2^{s-127} over blocks of 32 values, without dense expansion (`eval/mxfp4.go: mx4.gather`), the microscaling format of [18]. Attention sinks and YaRN are handled by the reference forward; the Champ kernels run neither, and no mixture. Router selections and weights are recorded per token into the flow record (`Experts`, `Gates`).

(G2) Update-expert admission gate. `hodgemix -update U -generate M` attaches U's feed-forward to every layer of M as a second expert on the same cells (`eval/eval.go: attach, consider, denseExpert, admit`). With z_0 the base FFN output and z_1 the update's (computed on the same normalized input and passed through the base's `ffn_sub_norm`), let $\Sigma_c(v) = \sum_{i \in \text{cell } c} v_i$ over the residual cells and $c^\star(v) = \arg\max_c |\Sigma_c(v)|$. The update is admitted for that token and layer iff

$$c^\star(z_0 + z_1) = c^\star(z_0) \quad \text{and} \quad \Sigma_{c^\star}(z_0)\, \Sigma_{c^\star}(z_0 + z_1) > 0, \tag{12}$$

and the FFN output is then $z_0 + z_1$ (gates $(1,1)$); otherwise z_0 (gate (1)). Without a cell layout the update is always refused. This is a hard gate on the residual cell moved, not a learned router.

(G3) Engine gate in the Champ page. (Carried from version 2: the page and `bitnet/main.go` are not in the `hodgemix-go` tree and were not re-checked.) `web/router.js` routes a chat message among four engines on a 2×2 grid (Lite, Balanced, Thinking, Pro+Think). Regular-expression and length cues give two probabilities, think p_t and size p_s, clipped to $[0, 1]$; the gate is the product $\pi_{(r,c)} = p_t^r (1 - p_t)^{1-r} p_s^c (1 - p_s)^{1-c}$, summing to 1. Selection is top-1 as in Switch [16]; engines not on the server are dropped and the rest renormalized, as in GShard [17]; a capacity margin 0.2 keeps the loaded engine when the winner leads it by less. With one model loaded, the same four gates choose each token's sampling temperature $(0.6, 0.85, 0.7, 0.95$; `bitnet/main.go:`

`mixTemperature`) and set a presence penalty $\min(0.3,\ 0.5(1-\pi_{\text{Lite}}))$, stacked by one logit per repeat up to 4 (`stackedPen`). A tool result enters the context only with positive cosine to the question. This gate acts on messages and sampling, not on weights.

2.8 BitLinear: weights as digit strings

A BitLinear $W \in \mathbb{R}^{n\times k}$ is stored as limbs $l = 0, \ldots, \Lambda - 1$:

$$W \;=\; \sum_l s_l\,(D_l - c_{r_l}\mathbf{1}), \qquad D_l \in \{0,\ldots,r_l-1\}^{n\times k}, \quad r_l \in \{2,3,4\}, \quad c_r = \lfloor (r-1)/2 \rfloor, \tag{13}$$

so a digit means $\{0,1\}$ for $r=2$, $\{-1,0,1\}$ for $r=3$ and $\{-1,0,1,2\}$ for $r=4$. Limb 0 is the BitNet ternary code ($r_0 = 3$, $s_0 =$ `weight_scale`); further limbs exist only with `-limbs`> 1 (default 1). The output is $y = \big(\sum_l s_l(D_l - c_{r_l})Q_8(a)\big)\cdot\sigma_a$ with σ_a the activation scale.

CPU: lookup-table coboundary. Each limb is a signed incidence operator on the activation cochain, evaluated by gathers and integer adds only (`eval/lut.go`), in the line of LUT-based low-bit kernels [7, 6]. The input axis of length k is cut into k/K blocks of K channels, where

$$K(r,k) = \max\{K : K \mid k,\ r^K \le 256\} \tag{14}$$

(`blockFor`); a row's block of digits $d_0..d_{K-1}$ is one byte, the numeral $c = \sum_t d_t r^t$. Per token and block b the table

$$T_b[c] = \sum_{t=0}^{K-1} \big(d_t(c) - c_r\big)\, a_{Kb+t} \tag{15}$$

is filled digit by digit (the levelized compile: after digit t, entries $[0, r^{t+1})$ hold all partial sums), and $y_r = \sigma_a \sum_l s_l \sum_b T_b^{(r_l)}[c_{rb}^{(l)}]$. Tables are cached per (radix, K) per input, so q, k, v share one set and `gate`/`up` another. Table 3 lists the block sizes for the 2B4T axes; every case tiles exactly. With one ternary limb the result is

Table 3: Block size K and table size r^K (`blockFor`).

axis	$r=2$	$r=3$	$r=4$
$k = 2560 = 2^9\cdot 5$	$K=8$ (256)	$K=5$ (243)	$K=4$ (256)
$k = 6912 = 2^8\cdot 3^3$	$K=8$ (256)	$K=4$ (81)	$K=4$ (256)

bit-identical to the integer multiply–accumulate (`TestLUTMatchesMAC`, re-run on the 2 November tree: pass, on synthetic sizes including 64×2560 and 16×6912), because the integer accumulator is exact and the float multiply order $(\text{acc}\cdot\sigma_a)\cdot s$ is preserved. The reported timings on a Ryzen 5 PRO 5650U: 334-token chat probe including load 3.5 min → 75 s; 81-token score ≈65 s → 36–43 s; 441-token calibration 270 s → 117 s. For prompt tokens whose logits are not read the head is skipped (`eval.go: feed`); the head is $|V|\times D$ BF16 per token, larger than all ternary layers together.

GPU: `bitlinear.wgsl`. (Carried from version 2; the `bitnet/` kernels and loader are not in the `hodgemix-go` tree and were not re-checked. Only `kernels/neuro.wgsl` is in the tree.) Champ reads the packed layout of Hugging Face BitNet checkpoints: U8 $[n/4, k]$, byte (r,j) holding the 2-bit codes of rows $r, r+n/4, r+2n/4, r+3n/4$ in bit pairs 0–1 … 6–7. A u32 is four inputs for four rows, and with the override constant `CENTER`$= c_r$ (default 1),

$$\textstyle\sum_j a_j(\text{code}_j - c_r) = \text{dot4}(a, \text{codes}) - c_r\,\text{dot4}(a, \texttt{0x01010101}), \tag{16}$$

32 lanes per packed row, 8 rows per 256-thread workgroup, followed by $(\cdot)\,\sigma_a\,s$. Limbs $l \ge 1$ are extra dispatches of the same pipeline with `ACCUM=1` and their own `CENTER` (`model/model.go: linOp`); a limb radix outside $\{2,3,4\}$ is rejected. With `CENTER`$= 1$ the kernel is byte-identical to the previous one. `champ.go` now checks limb tensors (`champLimbs`) and notes that `model.go` and `bitnet-rt.js` run every limb with no limb-count cap. The README still states that `model.go` needs `compile.bat`; whether the deployed `bitnet.wasm` has been rebuilt is not recorded in the tree. `flow.wgsl` reads the same packed layout and adds each product $a_j(\text{code}_j - c_r)$ exactly in i32 into its (row cell, input cell) bin; `flowscale.wgsl` multiplies by $\sigma_a s$ (§4).

2.9 Head

The head is tied: logits = $E\,\mathrm{RMS}_{g^{\mathrm{fin}}}(x)$ in BF16 (`eval.go: logits`; Champ `headW` is the embedding when `Tied`). Champ rejects `tie_word_embeddings=false` (`champ.go`); this is why the Hadamard residual rotation of [1] (which unties the head) is off by default.

2.10 File format and loading

Table 4: Tensor keys of a hodgemix output (per layer ℓ; `P`=`model.layers.`ℓ`.`).

Key	dtype	shape
`model.embed_tokens.weight[r0:r1]`	BF16	$[r_1 - r_0,\, D]$, $r_1 - r_0 \le 8192$
`P input_layernorm.weight`, `P post_attention_layernorm.weight`	BF16	$[D]$
`P self_attn.attn_sub_norm.weight`	BF16	$[Hd_h]$
`P mlp.ffn_sub_norm.weight`	BF16	$[F]$
`P self_attn.\{q,k,v,o\}_proj.weight`	U8	$[Hd_h/4, D], [H_{kv}d_h/4, D]^{\times 2}, [D/4, Hd_h]$
`P mlp.\{gate,up,down\}_proj.weight`	U8	$[F/4, D]^{\times 2}, [D/4, F]$
`X.weight_scale` (each BitLinear)	BF16	[1]
`X.weight_limb`i ($i \ge 1$, optional)	U8	as `X.weight`
`X.weight_limb`i`_scale` / `_radix`	BF16 / I32	[1] / [1]
`model.norm.weight`	BF16	$[D]$

The BNZ4 container (`bnz4/`) is the magic `BNZ4`, a u64 index length, a JSON index, and chunks with method $\in \{0, 1, 4, 5\}$ (raw, and rANS variants decoded in WGSL); an Ed25519 signature over header and index is written to `OUT.sig` when `-sign-seed` is given. Also written: the per-tensor signed invertible channel-map chunk of [1] §6 (`chanmap.go`), and `config.json` (`topology.go: configFor`). That file is the run file's output config, or else the base's, with the sizes the output tensors have; `model_type`, `architectures` and the activation stay. A missing `hidden_act` is copied from `hidden_activation`, keys the class leaves out take its defaults, and no `quantization_config` is added. `promoteRope` copies `rope_parameters` to `rope_scaling` when only the former is present. `champ.go` checks the C1–C10 requirements (shapes, multiples of 4, tied head, θ) before the container is accepted.

Hodge names (hkruntime). The runtime target is hkruntime, which reads only the Hodge names. After REPAIR and before the Champ check, `hodgename.go: stampHodgeFile` rewrites the run's output `config.json` (the run file's `config` entry for the output, `of.Config`) in place through `stampHodge`:

- `model_type` `bitnet` becomes `hodge`;
- for a ternary-family model (`bitnet` or `hodge`), `architectures` becomes `[HodgeForCausalLM]`;
- a `quantization_config` with `quant_method bitnet` or `linear_class autobitlinear` becomes `hodge`/`hodgelinear`;
- an `auto_map` that points at a `bitnet` class is dropped.

Each change is logged as `config.json: ...`. Tensor names are unchanged; the 2B4T tensor keys of Table 4 are what hkruntime loads.

Tower tensors. The output's only tower tensors are those `writeTowers` builds on the base embedding (§3.11):

- `bnzmix.vision_tower.*` and `bnzmix.audio_tower.*`: the L-A operator constants `sigma`, `zstar`, `lambda`, `ell`, `c`, `s`, `radius` and `depth`, plus `base_sha256`;
- `bnzmix.embed_vision.*` and `bnzmix.embed_audio.*`: `mean_feature` $[d_f]$, `mean_row` $[D]$, `rms` [1], and the projector `weight` $[D, d_f]$.

The projector is ternary with `weight_scale` and limbs, or F32 when D is not a multiple of 4. No encoder tensor of any input and no `bnzmix.embed_bridge` is written. All `bnzmix.*` names are ignored by the Champ loader (C7).

Vocabulary float file. When BNZ output is on, the run also writes `vocab.float.safetensors` beside the model (`vocabfloat.go`, §4). The checkpoint embedding is not changed.

BNZ4 and Champ (legacy). The BNZ4 container and the Champ loader requirements C1–C10 have been legacy since 21 October 2026. They are still produced and enforced. The ternary safetensors file is the primary output, and STAGE 5/6 and 6/6 still compress it to BNZ4 and verify the round trip; a signature follows when `-sign-seed` is given (§3.15).

2.11 Calibration set

Every data-dependent stage uses the base's reference forward on a fixed calibration set (`eval.go: calibration`). The set is six evaluation sentences, two greedy prompts and the chat prompt, which came to 441 tokens on 2B4T. The chat prompt is now rendered in the base tokenizer's own turn format (`eval/chat.go`: Llama 3 headers, Qwen `im_start`, 2B4T "Role: content"), so the token count depends on the base. One pass feeds surgery and pin-mean (`FindSingular`); repair runs its own passes (`NormStats`).

3 The assimilation pipeline

Table 5 gives the stage order of `main.go` (2 November) with the log line each stage prints, in the order `run` executes them. Figure 2 draws the same order. Equations of the core (index, spline grid, Hodge split) are those of [1] and [2] and are only summarized. A stage marked *legacy* belongs to the Champ/BNZ4 path and still runs.

Table 5: Stage order of `main.go` (2 November 2026), with log names.

Log name	Work	Source
(header)	`hodgemix: N CPU cores`; `loading base header`, `loading source header`. A topology other than the base's logs that surgery, pin-mean, repair, the OV chart and the union vocabulary are not run	`main.go`
`PREFLIGHT`	run-file checks before any write, then `PREFLIGHT ok: ... the output config runs in Champ`; `MRH base hidden D -> radices`	`preflight.go`, `mrh.go`
`CALIBRATE`	base reference forward on the calibration set; massive channels (only if surgery, pin-mean or repair is on, base topology)	`eval.go: calibration`, `FindSingular`
`VOCAB`	union vocabulary and residual map (base topology, at least one source)	`vocab.go`
`STAGE 1/6`	index entries + MRH keys; aside tensors and, for Gemma 4, keys outside `model.language_model.` skipped	`main.go`, `mrh.go`
`STAGE 2/6`	shared channel index, topology, Champ layout notes (legacy)	`topology.go`, `attention.go`
`STAGE 3/6`	per tensor: married axes, load + spline grid → seed (phase correlation + gap selection) → spectral Hodge → curl TIES → window lock → `train: one Ricci-DeTurck step` → base residual → pin-mean → ternarize + limbs → pack, channel map	`synth.go`, `dtft.go`, `train/`, `limbs.go`
`TOWERS`	`N encoder and per-layer tensors of the inputs are not written` (only if $N > 0$); then always `writeTowers: TOWERS L-A depth 4, embedding layers into residual width D`, vision and audio probes	`main.go`, `towers.go`, `tower/`
(index)	`bnzmix.index.entries` and entry metadata	`main.go`

Log name	Work	Source
`STAGE 4/6`	write ternary (and optional merged) safetensors; auxiliary files; union tokenizer; output `config.json` by `configFor` (with `promoteRope`); first run file `OUT.yaml`	`main.go`, `topology.go`, `runfile.go`
`OV CHART`	v to base chart, o first-order circuit (base topology)	`ovchart.go`
`SURGERY`	base codes on the singular set (base topology)	`surgery.go`
`REPAIR`	norm gains back to base statistics, `until mean |log2| spread is under 0.2` (base topology)	`repair.go`
`config.json: ...`	Hodge names stamped on the output config	`hodgename.go`
`Champ compatibility: OK`	fatal `champCheck` (legacy), with notes	`champ.go`
`LLAMA3`	neck pack and its `champCheck`, llama3-RoPE bases only	`neck.go`
(run file)	`OUT.yaml` rewritten with the served models, neck first if present	`runfile.go`
`STAGE 5/6`	BNZ compress (legacy); not run under `-bisect`	`bnz4/`
`STAGE 6/6`	BNZ round trip (legacy); skipped by `-no-verify`	`bnz4/`
`VOLUME`	`volume.safetensors` from the served ternary file; then `vocab.float.safetensors` (fatal on error)	`volume.go`, `vocabfloat.go`
(sign)	Ed25519 `OUT.sig`, only with `-sign-seed` (legacy)	`main.go`
`EVAL`	base vs output, CPU reference forward, prompts, texts and the page's first answer	`eval/`
`DONE`	output, tensor count, report path	`main.go`

OV CHART, SURGERY and REPAIR run only when the output has the base's topology.

load headers → PREFLIGHT → CALIBRATE → VOCAB → STAGE 1/6 keys → STAGE 2/6 index, topology, layout
→ STAGE 3/6 per tensor: smoothed 1-form grid → **seed: shift + gap selection** → Hodge split → curl TIES
→ **window lock** → **one Ricci–DeTurck step** → base residual → pin-mean → ternarize → limbs
→ **TOWERS** (inputs' towers not written; L-A towers + projectors on the base embedding) → STAGE 4/6 write, config, run file
→ OV CHART → SURGERY → REPAIR → Hodge names → [legacy: Champ check → LLAMA3 neck] → served run file
→ [legacy: STAGE 5/6 BNZ → STAGE 6/6 round trip] → volume → **vocab float** → [sign] → EVAL

Figure 2: Pipeline order (`main.go`, 2 November). Bold marks the stages that changed after 25 October. Brackets mark the legacy Champ/BNZ4 path, which is still executed.

3.1 Index, grid and alignment seed

Each tensor's rows and columns are ordered by the multi-radix heap key (`math.go: heapKey`). With `-index married`, all contributors' channels share one MRH per axis. The weights are sampled into a grid $G_r \times G_c$ by the smoothed 1-form of §3.2; the grid sides are products of whole radix levels (`splineGrid`, `radices`).

Seed by gap selection. The Re-Basin assignment [9] and its Hungarian solver are deleted. The seed used by both locks is `math.go: placeSource`:

1. `seedShift`: the per-digit phase-correlation cyclic shift of [1] Eq. (12)–(13).
2. `seedAssign`, which is `train.Align` (`train/select.go`). It is applied to rows, then to columns of the result. Each source row (column) is a fibre.
   - δ is the largest distance from a source fibre to its nearest base fibre.

Table 6: Flags of the pipeline and their defaults (`main.go`, 2 November). Run files set them under `settings`; `runs/hodge-4-instruct.yaml` and `runs/hodge-3.5-instruct.yaml` set `limbs: 2` and `src-rope-theta: 500000`.

Flag	Default	Effect
`-lock`	`dtft`	band lock: Alg. D with window certificate, or `v2` (`gapLock`) for comparison
`-index`	`married`	one MRH per axis node across contributors
`-seed-align`	`auto`	phase-correlation shift + gap selection (help text still says Re-Basin)
`-vocab`	`union`	union tokenizer and appended rows
`-topology`	`base`	output shape (`base`, `max`, `married`); `-target-config` overrides
`-attn`	`on`	QK-norm fold, RoPE marriage, logit temperature
`-rotate`	`off`	Hadamard residual rotation (refused by preflight)
`-base-residual`	`on`	restore base detail $(I - PR)W_b$
`-limbs`, `-limb-tol`	1, 0.15	body limb budget and tolerance
`-ov`	`chart`	OV chart
`-pin-mean`	`on`	rank-one mean-response pin
`-surgery`, `-surgery-ratio`	`on`, 20	surgery cap, massive-channel ratio
`-repair`	`on`	norm-gain repair until spread $<$ 0.2; `-repair-passes` is removed
`-mrh`, `-grid-max`	`dynamic`, `GridMax`	MRH levels, grid bound
`-band-inclusive`, `-seq-axis`, `-persistence`	off, `auto`, `on`	Alg. D band, sequence axis, heap-key persistence
`-g-theta`	4	enters only the population prediction K/K_c (report)
`-g-omega`, `-n-lost`, `-omega-update`	8, 255, `on`	*not read* by either lock; `-omega-update` is only logged
`-sign-seed`	(none)	Ed25519 signature of the BNZ4 (legacy)
`-eval`, `-bisect`	`on`, off	final evaluation; ternary-only scoring

- A base slot i is *certified* under two conditions. First, its nearest other base fibre is farther than $\sqrt{2}(1+\sqrt{n})\,\delta$ (`GapNeed`, Lemma 4.1(b) of the lattice paper, with n the number of fibres). Second, exactly one source fibre lies within $\sqrt{2}\,\delta$ of it.
- Two slots claiming one source are both decertified.
- A certified slot receives that source fibre. An uncertified slot receives the *base* value (the "cone cap", Theorem 8.2 at $\rho = 0$, as cited in the code).
- There is no assignment search, and the stored permutations stay bijections.

3. If an axis certifies nothing (zero row hits or zero column hits), the cone cap would make the whole tensor the base. `placeSource` then keeps the shifted source grid whole with identity permutations, and logs `seed: gap selection certified rows a/b cols c/d; source grid kept`.

The seed log line reads `seed (phase-corr per digit + gap selection)`. The seed is still per tensor, so connected weights can receive different maps. How often gap selection certifies fibres on real checkpoints is not recorded; $\sqrt{2}(1+\sqrt{n})\,\delta$ is about 24δ at $n = 256$. The `-seed-align` help text and `DECISIONS.md` decision 6 still name Re-Basin.

3.2 Smoothed commuting 1-form on the grid

Up to version 2, `splineGrid` sampled each axis with a Whitney hat at the cell centres. It now projects each sorted axis onto the smoothed commuting 1-form of `fec.md` §6 (`math.go: splineGrid, newAxisGeom`). On an axis of G coarse cells, $h = 1/G$, with mollifier half-width $\varepsilon = h$ (one coarse cell, $\delta = 1$), let Q be the box (sum-preserving cell integral) of the fine values u onto the coarse 1-forms and M the mass matrix of the coarse

1-forms (`factorMass`, an LU of Q restricted to them). Then

$$\sigma = M^{-1} Q\, u, \qquad \mathrm{out}_c = v_c + \tfrac{h}{2}\,\sigma_c, \tag{17}$$

with v_c the cell value. Rows are projected first (one column at a time), then columns. It is the bounded commuting projection of `fec.md`; its harmonic-error and lock measurements there are on synthetic tensors only. Only `splineGrid` changed: `sampleGrid` (grid back to channels) and `newAxisInterp` still use the hat. `fec.md` still records the construction as "not approved"; the code ships it regardless, which §7 lists.

3.3 Hodge split and the window lock

In the mixed-radix DFT domain of the grid (`fftn`), the field splits into gradient, curl and harmonic parts (`Spectrum.Hodge`). With `-lock dtft` (the default), `dtft.go: synthD` runs Algorithm D of [2] up to its step 4:

- band edge $2 - 2\cos(2\pi/r)$ and frequencies from $(K - \omega^2 M)v = 0$;
- Hann window, Candan/Jacobsen sub-bin estimates and a Goertzel phase at the base frequency;
- the PSR/uniqueness gate and the L2 pair bound 1/128.

The curl set outside the band and gradient set is merged by ternary TIES (§3.7) before the band is decided.

Step 5 is now a certificate, not an anneal. The Kuramoto/KTE tick loop and its coupling matrices are removed from `synthD`. A band oscillator k of contributor a is *clear* if its measured phase already lies within the lock window of the base mode m:

$$\big|\,\mathrm{lift}(\theta_k - \theta_m)\big| \le \tfrac{1}{16}\ \text{turn}, \qquad \mathrm{lift}(x) = x - \mathrm{round}(x). \tag{18}$$

The order parameter $\bar r$ is the mean over modes of $|\frac{1}{|A|}\sum_a e^{2\pi i\theta_{a,m}}|$, computed once, and the step count is reported as 1. The comment states the intent: a sheet outside the window is not dragged in by a tick loop. The merge decision then applies the four gates (i) PSR, (ii) L2 pair bound, (iii) winding and (iv) window, and merges a mode over its members by Eq. (19). The log reports `window-locked=n/m`, where it used to report `tick-locked`. The population prediction K/K_c (`popPredict`, gain $2^{-\texttt{g-theta}}$) is still computed for the report only.

With `-lock v2` (comparison only), `synth.go` uses `gapLock`. A phase is locked when $|\,\mathrm{wrap}(\theta_i - \theta_{\mathrm{ref}})| \le W_{\mathrm{lock}} = 2\pi/16$, with one step and `Converged` true. Phases are not integrated.

3.4 One Ricci–DeTurck step (package `train`)

Every tensor that leaves `synthD` or the v2 `synth` takes one step of `synth.go: flowGrid`, which is `train.Flow(out, ref)` with `ref` the base grid (`train/ricci.go`, `train/hardy.go`). This holds for the locked case, the no-band case and the no-lock case alike. With $f = \mathrm{out} - \mathrm{ref}$ on the $G_r \times G_c$ grid treated as a torus:

$$f_i' = f_i + \Delta t\,(\Delta f)_i - \Delta t\, c_H\, \Omega(x_i)^2 f_i, \qquad \mathrm{out}' = \mathrm{ref} + f'. \tag{19}$$

The terms are:

- Δ is the periodic five-point lattice Laplacian (`Lap2`), and $\Delta t = 1/8$ (`DT`, the 4-lattice mesh constant).
- $c_H = 2 - 4/\rho^2$ is the Hardy constant (`CH`), and $\Omega(x) = 2\rho/(\rho^2 + |x|^2)$, with x the site's offset from the grid centre.
- $\rho = 531/40$ (`Rho`).

Neck cap. A site with $|(\Delta f)_i| \ge 6/\rho^2$ (about 0.034) is a lattice neck. Its residual is set to 0, so it takes the base value. The count is logged as `train: one Ricci-DeTurck step, neck sites capped=N`.

The step is not repeated, and its size is not searched. A residual with $\max|f| \le 10^{-12}(\max|\mathrm{ref}| + 1)$ is returned unchanged. The threshold $6/\rho^2$ is absolute, so whether a site is a neck depends on the scale of the tensor's weights; no run records the capped counts. This is a single explicit step of the scalar principal part plus a Hardy potential, as the code comment says. It is not a gradient step and uses no data. `train` also holds `Chern2`, `ConeCap`, `StripDelta`, `BranchLevel`, `SelectAngles` and `FactorRadius2`, which the pipeline does not call.

3.5 No band, no lock

`synthD` distinguishes the following cases:

- *No band, with sources.* The log is `[D] no band to lock: curl residual kept for the Ricci-DeTurck step`. The tensor keeps its curl-TIES result and takes the step of §3.4.
- *No mode locked.* The log is `[D] no mode locked: band stays with the base; curl residual kept for the Ricci-DeTurck step`. The band bins stay with the base by Eq. (19), the curl residual stays, and the step follows.
- *No band and no sources*, or no digit with $r \geq 3$. The band stays with the base and there is no lock.

The 25 October fallback that replaced such a tensor by the base grid is removed. The code comment gives the reason: the replacement made the step a no-op, so an unlocked Llama tensor shipped as the base.

3.6 Anneal early stop: limb lock only

`kuramoto.go: runAnnealStage` is unchanged:

- it skips the stage at $r_0 \geq 1 - r_{\text{tol}}$;
- it stops at $r \geq 1 - r_{\text{tol}}$;
- it stops after `annealStall`$= 8$ ticks without a rise of $r_{\text{tol}} = 2^{-8}$;
- it stops at tick $n/2$ when the best r is below $1 - 16r_{\text{tol}}$.

It is now reached only through `kuramotoTick` in `limbs.go: lockLimb`, with constant gains 2^{-4}, $N_{\text{lost}} = 255$, and through the self-test. `lockLimb` runs only for body limbs, that is with `-limbs`> 1. The band lock of both `-lock` modes has no anneal.

3.7 Curl set: ternary TIES over all contributors

Outside the band and gradient set, each contributor's delta δ_k from the base is coded at its absmean, $c_{k,i} = \text{RoundClip}(\delta_{k,i}/\overline{|\delta_k|})$, the sign is elected $\sigma_i = \operatorname{sgn} \sum_k w_k c_{k,i}$, and (`math.go: ternaryTIESAll`, called from `dtft.go` and `synth.go`)

$$\tau_i = \frac{\sum_{k:\, c_{k,i} = \sigma_i} w_k \delta_{k,i}}{\sum_{k \in \text{all contributors}} w_k}, \qquad W \leftarrow W + \tau, \tag{20}$$

with weights normalized over every contributor including the base (total 1). With one source at 1:1 this equals version 1's $\lambda = 1 - w_{\text{base}}/\sum w$ rule ($\frac{1}{2}\tau$); with several sources, or the base repeated as a source, it differs: zero-delta contributors dilute toward the base (`TestTIESAllWeights`). The former `ternaryTIES` remains in the source, unused by the merge.

3.8 Base residual, scale, pin-mean

The base's detail finer than the index is restored, $W \leftarrow W + (I - PR)W_b$ ([1] Eq. (22)). A tensor the base has keeps the base's absmean scale (`main.go`). Then, for base-shaped tensors without rotation, *pin-mean* (`pinmean.go`) removes the change of the response to the mean input:

$$W \leftarrow W - (\Delta W \hat{\mu})\hat{\mu}^\top, \qquad \Delta W = W - W_b, \quad \hat{\mu} = \mu/\|\mu\|, \tag{21}$$

with μ the base forward's mean BitLinear input over the calibration tokens: `in_attn` for q, k, v, `in_o` for o, `in_ffn` for gate/up, `in_down` for down. This is a rank-one projection per tensor that makes $\Delta W \hat{\mu} = 0$.

3.9 Ternarization, body limbs and neck limbs

Codes are $q = \text{RoundClip}(W/\bar{s})$ with stored scale $s_0 = \sum_{q_i \neq 0} |W_i| / \#\{q_i \neq 0\}$. `residualLimbs` then works on the gap $\Delta = W - s_0 q$:

1. *Neck.* $N = \{i : |\Delta_i| > \tau(n)\,\mathrm{median}\,|\Delta|\}$ with the T1 threshold of [3], $\tau(n) = \sqrt{2\ln(n/2^{-8})}/\Phi^{-1}(3/4)$ (`t1Tau`, `singularNeck`), where $n = R \cdot C$ is the number of entries of the tensor. The neck is zeroed in the field the body limbs are fitted on.
2. *Body limbs* (up to `-limbs`-1, while the relative residual exceeds `-limb-tol`): radix and step as in version 1 (`quantLimb`); each limb is then Kuramoto-locked against limb 0 on the harmonic bins of the channel grid (`lockLimb`, [1] Eq. (14)–(20)); a bin that does not lock stays with limb 0; digits on N are set to c_r; a limb that does not reduce the residual ends the loop.
3. *Neck limbs*: `NeckLimbs`$= 4$ further limbs code the remaining gap on N only, each with its own scale, no lock, body at c_r (`neckLimbs`). They are written on every run where $N \neq \emptyset$, independent of `-limbs`.

For a base whose RoPE type is `llama3` (`eval.RopeType` of `eval.RopeDict`, i.e. `rope_scaling` or `rope_parameters`) the body budget is 6 limbs at tolerance 0.02 (`Llama3Limbs`, `Llama3Tol`); the packed file is `<out>.neck.safetensors`, built from the ternary file and the float checkpoint by `neck.go: packNeck`; `appendSing` appends neck limbs to a file written before this change, without re-locking.

3.10 Limb stopping rules

`residualLimbs` (`limbs.go`) has three new exits, each with the BF16 refine tolerance `RefineTol`$= 2^{-8}$:

1. *Body.* A body limb is kept only if it lowers the body part of the gap (entries off the neck) by at least 2^{-8}; otherwise the loop ends. The comment records `hodge-4` tails of 10^{-7} after the residual had stopped, all accumulated by the loader.
2. *Wide neck.* If the T1 neck mask covers more than 1/5 of the entries (`neckWide`) it is not a spike tail, and no neck limbs are written; on `hodge-4` the mix residual marked about half of every weight and the loader accumulated that limb at 0.25 of the base RMS.
3. *Neck.* A neck limb is kept only if it lowers the neck part of the gap by at least 2^{-8}.

`MaxLimbs`$=$ 1 and `LimbTol`$=$ 0.15 remain the defaults; `runs/hodge-4-instruct.yaml` and `runs/hodge-3.5-instruct.yaml` set `limbs: 2`. The comment on `MaxLimbs` still cites ppl 4142 against 14.4 on 2B4T + Qwen3 at two limbs (README), measured before the neck cut and the limb stops.

3.11 TOWERS stage

After the STAGE 3/6 loop and before STAGE 4/6 writes the file, `main.go` does two things. Nothing from an input's towers reaches the output.

T1 *Count the aside tensors.* The aside tensors (§2.5) of the base and of every source are counted, and when there are any the stage logs `TOWERS N encoder and per-layer tensors of the inputs are not written; the output's towers are the L-A towers`. The report key is `not_written`.

T2 *writeTowers* (`towers.go`) runs on every run. It is skipped only when the base has no token embedding, and then logs `TOWERS skipped: the base has no token embedding...`. Otherwise it logs `TOWERS L-A depth 4, embedding layers into residual width D`, with D the base embedding width.

The L-A operator. The vision and audio towers are built on the L-A operator of decisions D13–D16 as cited in `tower/la.go`:

- $T_1 = \sin(c/W_0(z))\sin z$ and $T_2 = s\sin z$, with $c = e^{i\pi e}$ and $s = -\sin 1$;
- the attracting fixed point is $z^\star = 1.6i$, and the stage stops if the operator has none;
- the seed disc radius is $r = 0.175$ and the depth is $n = 4$ (`tower.Radius`, `tower.Depth`);
- the readout is Koenigs, $R_n = \lambda^{-n}(z_n - z^\star)$;
- the feature widths are $d_f = 46$ for vision and $d_f = 240$ for audio.

The projectors. Each projector $P \in \mathbb{R}^{D \times d_f}$ is fitted in closed form by least squares (`fitProjector`):

- $n = \min(d_f + 32, |V|)$ synthetic probe features (`probeCount`) are paired with the base embedding rows $i(|V|-1)/(n-1)$, $i = 0..n-1$ (`sampleRows`);
- after centring, the normal equations get a ridge $10^{-6} \operatorname{tr} A/d_f$ and are solved by `cholSolve`;
- P is ternarised with absmean codes and residual limbs as any other tensor (D7, `residualLimbs` at `MaxLimbs`, `LimbTol`), or stored F32 when $D \not\equiv 0 \pmod 4$;
- the mean feature, mean row and row RMS are stored with it.

The output names are listed in §2.10, and the base header SHA-256 is included (D9). The report key is `towers`. `tower/` and `towers.go` are unchanged since 25 October apart from the removal of `copyTowers`.

What the code does not do. No gradient step is taken on the towers. Two points differ from the decisions as cited in the code comments:

- D2 specifies towers married to the residual axis and run through the same lock/TIES path, while the code fits the projector by least squares outside the lock.
- D6 specifies pairing with base rows of *paired text tokens*, while the code pairs synthetic probes with evenly spaced rows.

The D10 acceptance tests are not in the code.

Width under a married topology. `writeTowers` takes D from the *base* embedding. Under `-topology married` the output residual width differs from that D. In the root test `TestMarriedTopologyRun` the towers are built at width 16 while the output config is written at hidden size 48. The projectors then do not match the output's residual width. This is an open issue (§8).

3.12 OV chart

For a head h in KV group g, mixing both factors of the circuit gives $(O_b + \Delta O)(V_b + \Delta V) = O_b V_b + \Delta O V_b + O_b \Delta V + \Delta O \Delta V$. The chart (`ovchart.go: ovChart`) removes the cross term:

$$V_t = V_b, \qquad O_{t,h} = O_{h,\text{mix}} + O_{b,h}\, X_g, \qquad X_g = \Delta V_g V_{b,g}^{+}, \quad V^{+} = V^{\top}(VV^{\top})^{-1}, \tag{22}$$

so $O_{t,h} V_{b,g} = O_{b,h} V_{b,g} + \Delta O_h V_{b,g} + O_{b,h} \Delta V_g P_g$ with P_g the projector on the row space of $V_{b,g}$. X_g is one $d_h \times d_h$ SPD solve per group, $X_g = K_g G_g^{-1}$, $G_g = V_{b,g} V_{b,g}^{\top}$ (Tikhonov $10^{-9} \operatorname{tr} G/d_h$), $K_g = \Delta V_g V_{b,g}^{\top}$ (`solveSPD`). It is applied to the written ternary file: $\Delta O, \Delta V$ are from dequantized codes, v is overwritten with the base's codes and scale, O_t is re-ternarized with the least-squares scale, v's limbs are zeroed and o's are refilled from O_t. The reported cross term is $\|\Delta O \Delta V\|_F / \|O_b V_b\|_F$ computed by Gram traces on the first head of each group.

3.13 Surgery cap

With the singular set $(\mathcal{M}, \{S_\ell\})$ of the base forward (§2), `surgeryWith` puts back the base's ternary codes on

- rows $m \in \mathcal{M}$ of o and down (writers) and columns $m \in \mathcal{M}$ of q, k, v, gate, up (readers);
- rows $u \in S_\ell$ of gate and up, and columns $u \in S_\ell$ of down (the FFN side).

Every tensor keeps its own `weight_scale`; limbs on capped rows/columns are set to the zero digit c_r. For a float base the base codes are its absmean ternarization (`baseCodes`). This corresponds to step 6 (cap and glue) of Algorithm S in [3], with the pathway set and detection rule described in §7. The construction is the discrete counterpart of Perelman's Ricci flow with surgery on three-manifolds [25, 26, 27]: the singular neck is cut out and a standard cap (the base pathway) is glued in, while the flow (the lock) runs on the remainder.

3.14 Repair

`repair.go` re-fits each RMSNorm gain that the base checkpoint itself stores: `input_layernorm`, `attn_sub_norm`, the FFN pre-norm, `ffn_sub_norm` and `model.norm`. The FFN pre-norm is `post_attention_layernorm` or, when that is the name the base stores, `pre_feedforward_layernorm`. With σ_c the RMS of channel c after the gain over the calibration tokens, each pass applies

$$g_c \leftarrow g_c \cdot \mathrm{clamp}\big(\sigma_c^{\mathrm{base}}/\sigma_c^{\mathrm{out}}, \tfrac{1}{2}, 2\big) \qquad (\texttt{repairRatio}). \tag{23}$$

The pass cap and `-repair-passes` are removed. The loop `for pass := 0; ; pass++` measures the output forward and stops at the first pass whose mean $|\log_2|$ spread is below 0.2 (`repairStop`). That pass writes nothing and logs `gains left as they are`. There is no upper bound on the number of passes, so a run whose spread never falls below 0.2 does not terminate in REPAIR.

Sub-norms that Champ inserted at unit gain for a base that has none are left at 1 (`baseOwnsGain`). The clamp is per pass, so a stored gain's cumulative factor is $2^{\pm P}$ for P executed passes.

3.15 Champ check (legacy)

The Champ/BNZ4 path was marked legacy on 21 October, but every piece still executes:

- `champLayout` in STAGE 2/6 (QK-norm drop, sub-norm insertion, tied head);
- `champRuntime` on the output config in preflight;
- the fatal `champCheck` after the Hodge stamp, and again on the llama3 neck file;
- STAGE 5/6 BNZ compression and the STAGE 6/6 round trip;
- the optional `-sign-seed` signature.

Three things changed since 25 October:

- `champRuntime` no longer requires the quantization config to be `hodge`, `hodgelinear` or `offline`.
- It accepts RoPE type `mrope`, and reads RoPE from `rope_scaling` or `rope_parameters` (`eval.RopeDict`).
- The stamp and `champCheck` read the run file's output config (`of.Config`) rather than a path beside the model.

`champCheck` checks the BitNet tensor shapes and limb tensors (`champLimbs`). It reports extra tensors (`bnzmix.*` and any other model tensors) as counted notes, not failures. A failure stops the run with `the output does not run in Champ`.

3.16 Run file and preflight

A run is described by a YAML run file (`runfile.go: loadRunFile`). It names each model's files (weights or shard list, tokenizer, tokenizer_config, config, optional special-tokens map and generation config), weights, output and settings. `config` is now *required* on the base, on every source and on every `models` entry; a missing one stops the run with `no config set (run file: config)`. Unknown keys also stop the run. `writeRunFile` takes a list of entries. It writes `OUT.yaml` after STAGE 4/6, and rewrites it after the Champ check with the served models, the neck file first when there is one (`bindServed` records them on the checkpoint the BNZ holds).

`preflight.go: preflightCheck` verifies before anything is written:

- configs describe their tensors;
- tokenizer ids lie within the embedding rows;
- `eos_token` is an added token;
- no tensor of any model is left without a role (biases, fused qkv, role-less 2-D tensors, and q/k norms with `-attn off` stop the run; aside tensors are exempt);
- the base head is tied, and `-rotate on` is refused;
- the output config passes `champRuntime`.

A mixture-of-experts source carries tensors with no merge role, so it cannot enter the merge.

The ordering bug of the first draft of this version is fixed. Preflight still runs `champRuntime` before the Hodge stamp, but `champRuntime` no longer tests the quantization names, so an unstamped `bitnet/autobitlinear` base config now passes.

Run files in the tree. `runs/hodge-3.5-instruct.yaml` (1 November) has a Llama-3.2-3B-Instruct base and a `hodge-vaultgemma-1` ternary source. `runs/hodge-4-instruct.yaml` (27 October) has its base files in `Downloads` and a `hodge-4` ternary source. Both set `limbs: 2` and `src-rope-theta: 500000`. Neither has a log made with the 2 November code. `runs/hodge-4-instruct.log` (27 October) predates the code: it shows `curl stays with the base` and `PREFLIGHT ok`.

4 Weight volumes and the forward cochain

Cells. `volume.go` groups every axis into cells shared by all tensors on that axis node: the residual axis D has one order for every reader and writer, the FFN axis F one for gate/up rows and down columns (MRH pop order of the averaged Eq. (7) keys of [1]); attention keeps heads whole (4 cells per head). For an MoE checkpoint the keys are averaged one expert at a time (`moeAxes`). Every run writes `volume.safetensors`: `volume.<role>` $[L, \text{out cells}, \text{in cells}]$ is the cell mean $|W|$ over the tensor's absmean, `balance.<role>` is mean W over mean $|W|$, `cell.<axis>` is each channel's cell; MoE layouts get per-expert volumes (`writeExpertVolumes`).

Currents. With the cells given, every BitLinear's digit columns are laid out in cell order with a LUT block size dividing the cell size, so each gathered table entry is the integer current from one input cell into one row; outputs remain bit-identical (`eval/flow.go`, `TestRelayoutCurrents`). The GPU path (`flow.wgsl`, `flowscale.wgsl`, `cellsum.wgsl`, `model/flow.go`) is in the `bitnet/` tree, outside `hodgemix-go`, and is carried from version 2.

Cochain and Hodge split. `cochain.go` builds per token a graph whose nodes are each layer's residual cells (entering, after attention, after the FFN) and the q, k, v, attention-output, gate, up and hidden cells, and whose edges are every voxel current plus the residual carries, attention's $v \to$output transport and up$\to$hidden (gate as metric). With d_0 the node-to-edge coboundary and a block-diagonal metric (each (layer, class) block in its own RMS units), the flow splits as

$$f = d_0\phi + f_{\text{cyc}}, \qquad L_0\phi = d_0^\top f, \quad L_0 = d_0^\top d_0, \tag{24}$$

solved by conjugate gradient (`hodge`). Each residual cell now also carries two 2-cells, digons of the net sub-layer write against the residual carry (attention, then FFN; `flowFace`). On a digon the coexact part is $(\gamma/2, -\gamma/2)$ with $\gamma =$ write $-$ carry, and the harmonic part (`harmonic`) is the mean of the two edges, with circulation 0. The log reports the face count, the harmonic share and the largest face circulation, and the per-token wedge of (7) is written as `t<pos>.wedge`. Diagnostics: conservation, Stokes ratio $\|d_0^\top f_{\text{cyc}}\|/\|d_0^\top f\|$, gradient share, hottest layer, moved residual cell, experts fired. Recorded on 2B4T for one prompt before the digons and the wedge gate (README): 15 664 nodes and 760 800 edges per token, conservation $2 \cdot 10^{-6}$, Stokes $9 \cdot 10^{-10}$, 188 CG iterations ($\approx$1.2 s per token), gradient share $\approx 7\%$ against 2.06% for random currents on the same complex.

Vocabulary on the residual cochain. After `volume.safetensors`, `vocabfloat.go: vocabFloat` reads the served embedding and the volume's `cell.D`. It writes `vocab.float.safetensors` with two tensors:

- `cochain` $[V, n]$, each row's cell sums $\Sigma_c(E[t])$;
- `float` $[V, D]$, the channel vector that gives back those sums, with each channel of a cell receiving an equal share $\Sigma_c/|c|$.

A row that is constant on each cell is returned unchanged. The checkpoint embedding is not modified. The step is fatal: a cell with no channel stops the run with `vocab: residual cell N holds no channel`, after the BNZ and volume are already written. Both married-topology root tests end there (§8).

5 Tools outside the pipeline

The following are in the 2 November tree and are not called by the merge. The wedge gate of §2.4 is the exception: it is in the reference forward and so affects CALIBRATE, REPAIR and EVAL.

- `itt/`: a separate `main` with two modes. `strip` writes an integer cochain safetensors from an instrumented model folder, and `text` builds the anchor chain and top cochain of one text. A served desk (`desk.go`, `desk.html`) mounts one instrumented model and its volume; the browser sends no path.
- `turn/`: a "mother and father" child circuit. The circuit is $C = BA$ (writer $\times$ chart). The turn is the orthogonal factor of $C_{\text{father}} C_{\text{mother}}^{\top}$, and the child is the sum carried into the halfway orientation, re-split into a writer and a chart. It is not imported by any other package.
- `eval/chat.go`: chat turns follow the checkpoint's tokens. These are the Llama 3 header turn, the Qwen `im_start`/`im_end` turn (including the Qwen-Image text encoder, with its own system line `ImageSystem`), Gemma's folded system turn, and the 2B4T "Role: content" form. `eval/converse.go` is a multi-turn conversation suite, and `eval/sample.go` a sampling session with `Segments`.
- `eval/neuro.go` and `kernels/neuro.wgsl`: graded-spike and non-adjacent-form evaluation of an int8 BitLinear input, bit-identical to `applyIn` (notes in `neuromorphic.md`).
- `towerwasm/`: the `tower` package for the page (§2.6).
- `stdiff`: it now also matches tensors by content SHA-256 across names.
- `splice_la_test.go` behind the build tag `splicela`. The `splice` and `spliceav` tower splice tests of 25 October are gone with the carried towers.

6 Measured behaviour (smoke test)

No number below was measured with the 2 November code. All of them predate gap selection, the window lock, the Ricci–DeTurck step, the removal of the no-lock fallback and the wedge gate in the reference forward. They come from the CPU reference forward on 81 teacher-forced tokens (six sentences) and one 334-token chat prompt. No two runs share a configuration. Results from grafted or post-patched copies, and the "Llama

Table 7: Reported results, 15–18 October 2026 (older code).

Run	ppl (81 tok)	note
2B4T base	12.0	reference, before the wedge gate
2B4T + Qwen3, defaults of 15 Oct	14.4	334-token chat answers
same, `-seed-align off`	$1.3 \cdot 10^7$	(earlier defaults)
same, `-limbs 2`	4 142	before neck cut and limb stops
stock 2B4T, all weights in a radix-4 limb (`TestLimbProbe`)	12.0	format path exact
Llama-3.2-3B-Instruct, BF16, reference forward	7.8	"Paris", $17 \times 23 = 391$
`hodge-2b4t-1` vs. 2B4T (`stdiff`)	–	542/542 tensors byte-identical

without repair" figure, are retracted and not used. The newest log in the tree, `runs/hodge-4-instruct.log` (27 October), ran the earlier no-lock fallback and is not evidence for the current code.

7 Implemented versus specified

Item	Specified	Implemented
Detection (Alg. S step 1)	T1 on residual channels ($\tau_{2560} = 7.67$), T2 int8 rule, T3 strict-majority writer, T4 curvature [3]	surgery cap unchanged: flat $\rho = 20$ (17 channels). T1 now used only for neck limbs, on weight-gap *entries* with $n = R \cdot C$ ($\tau \approx 9.6$–9.9), not on channels. T2–T4 unused.
Pathway set (step 2)	writers and readers at $\mathcal{M}$, writer units in ℓ_w only, norm gains at c, embedding column c	writers and readers at $\mathcal{M}$; spike units in *every* layer that has them; gains and embedding column not capped (repair touches all gains).
Pin seed (step 3)	$\pi(\mathcal{M}) = \mathcal{M}$ pointwise	not implemented: gap selection may copy a source fibre onto $\mathcal{M}$ before the cap.
Cut and fill (step 4)	sources filled with base on $\mathcal{S}$ before the lock	not implemented.
Flow on remainder (step 5)	drop oscillators supported in $\mathcal{S}$	window certificate on the full grid, then one Ricci–DeTurck step whose lattice-neck cap writes the base; the limb lock excludes the neck entries N.
Cap (step 6)	base trits on $\mathcal{S}$, with $\gamma_t = \gamma_b$ or difference absorbed in gains	implemented; capped trits use the output tensor's scale, neither option (i) nor (ii) applied explicitly.
Share repair (step 7)	gains of $U^\star$ and $C^\star$ only, $g \leftarrow g(\varepsilon^b/\varepsilon^o)^{1/2}$, band $[\frac{1}{2}, 2]$	every channel of every base-stored norm, ratio of RMS (exponent 1), clamp per pass, passes until spread < 0.2 with no cap.
Verify (step 8)	re-detect on output	not implemented.
OV chart, pin-mean, union vocab, limbs, LUT forward	no equation in [1, 2, 3]	implemented as in §3; their equations appear only in code comments and this paper.

Against the monodromy/MoE construction [14].

Piece	Status in code
Router tensor per layer	read and run for gpt-oss checkpoints in the reference forward (G1); never written into a hodgemix output.
Per-expert FFN tensor sets	read (float or MXFP4) for gpt-oss; per-expert volumes; not written; preflight gives them no merge role.
Top-k kernel	CPU only (`topkSoftmax`); no WGSL top-k, gather or scatter.
BNZ4 index entries for experts	none.
Champ loader support	refused: `champRuntime` rejects MoE `model_types`; `model.go` has no expert path.
Experts = sources kept separate	G2 attaches one source's FFN as a second expert, admitted per token by (12); reference forward only.
Static per-layer routing	not implemented.
Holonomy, depth monodromy $\mathcal{T}$, defect η, Δ_{RH}	not computed; the seed's per-tensor permutations are not exported in a form from which they could be.
Averaging-vs-experts criterion	not implemented; G2's criterion (moved residual cell and sign) is not η.
Message-level gate	G3 names the monodromy construction as its MoE gate; it routes messages among engines and sets sampling temperature, outside the weight-bundle construction of [14].

Piece	Status in code
One frame per axis node	implemented for the cell layout (`volume.go`), the single-frame gauge of [14] Prop. 1; the merge seed still aligns per tensor.

Version 3 items (2 November code).

Item	Specified	Implemented
Multi-Radix Geometry Thms 6.12, 6.13 (with Prop. 6.13B), 6.14 [12]	merge-improvement criteria in terms of a, q, $\lambda^{\star}_{\omega}$	**paper-only**: no code computes a, q or $\lambda^{\star}_{\omega}$. `train` takes one fixed explicit step and tests no criterion.
Gap selection	Lemma 4.1 of the lattice paper (`Lattice_Hilbert21_MRH`, cited in code)	`train.Align` as in §3.1; the fallback that keeps the shifted source when an axis certifies nothing is the code's own, not the lemma's.
Ricci–DeTurck step	`discrete_ricci_flow_M4`, Hardy constants (cited in code)	one step, $\Delta t = 1/8$, $\rho = 531/40$, neck $6/\rho^2$ absolute; not repeated.
Lock	Alg. D step 5 KTE population lock [2]	window certificate; `-g-omega`, `-n-lost`, `-omega-update` not read.
Smoothed 1-form grid	`fec.md` §6, marked "not approved"	in `splineGrid`; `sampleGrid`, `newAxisInterp` still the hat.
Towers married (D2)	projector axis married to D, same lock/TIES path	least-squares fit outside the lock, at the base embedding width.
Probes (D6)	base rows of paired text tokens	synthetic probes, evenly spaced rows.
Acceptance (D10)	ppl ratio ≤ 1.06, NLL, Class II, $N_c > n$, injective readout	not in code.
RoPE phase (D22)	planes 56–63 turn by pos $f_p + \phi_t$	in the reference forward; not in Champ.
Wedge gate	none	in the reference forward, unconditional (§2.4).
Seed documentation	`DECISIONS.md` decision 6, `-seed-align` help: Re-Basin	stale: the code is gap selection.
`TRACE.md`	Eq. (14)–(18) rows cite `kuramotoTick`; R-KTE-1 `kteTick` row	stale for the band lock; `kuramotoTick` remains only in `lockLimb`, `kteTick` only in `pairLag`.
README opening	"towers on a contributor are carried into that file"	contradicts the code and the README's own audio/vision section: nothing is carried.
`TestConfigFollowsTensors`	output `model_type`	stale: expects `bitnet` from a path beside the model.

8 Open issues

1. **Vocab-float stops married runs.** `TestMarriedTopologyRun` and `TestMarriedTopologySelfIsOnePermutation` end the test binary through `os.Exit` at `vocab: residual cell N holds no channel`. Under `-topology married`, `cell.D` of the volume names a residual cell with no channel, and `vocabFloat` is fatal. This happens after the BNZ and the volume are written. With those two tests skipped, the root package gives 52 pass, 8 skip and 1 fail.
2. **Tower width under married topology.** `writeTowers` fits the projectors at the base embedding width D (16 in the test), while the output's hidden size is the married width (48). The `embed_vision`/`embed_audio` projectors then do not match the output residual axis.

3. **Wedge gate in the reference forward.** It is unconditional. It changes calibration, the repair statistics and every evaluation, including of stock models. No ppl has been recorded with it, and it has no paper equation.
4. **REPAIR has no pass cap.** The loop ends only when the mean $|\log_2|$ spread falls below 0.2. A run whose spread stalls above it does not terminate.
5. **Gap selection unmeasured.** How many fibres Lemma 4.1 certifies on real checkpoints is not recorded. When an axis certifies none, the shifted source is kept whole, so the seed then reduces to the phase-correlation shift.
6. **Ricci–DeTurck neck threshold is absolute.** $6/\rho^2 \approx 0.034$ is compared with the lattice Laplacian of the weight residual in the tensor's own units, so the capped fraction depends on the weight scale. No capped counts are recorded.
7. **Dead flags.** `-g-omega`, `-n-lost` and `-omega-update` are not read by either lock, and `-g-theta` enters only the report's K/K_c. The `-seed-align` help still says Re-Basin. The STAGE 3/6 log line still reads "Kuramoto lock" though the band lock is the window certificate.
8. **Stale documents.** The README opening still says contributor towers are carried. `DECISIONS.md` decision 6 says Re-Basin. `TRACE.md` still maps the band lock to `kuramotoTick`/`kteTick`. The tower decisions D1–D24 cited in code live in `transfractal_tensor.tex`, which is not in the tree.
9. **Stale test.** `TestConfigFollowsTensors` expects `model_type bitnet`.
10. **No run with this code.** The newest log (27 October) predates gap selection, the window lock, the Ricci–DeTurck step and the wedge gate. No number in §6 applies to the current code.
11. **Towers vs decisions.** D2 and D6 differ from the code, and the D10 acceptance tests are absent.
12. **Smoothed 1-form not approved.** `fec.md` §6 is still marked "not approved" while `splineGrid` uses it.
13. **Champ/BNZ4 legacy but executed**, including the fatal `champCheck` on every run.
14. **Multi-Radix Thms 6.12–6.14** are paper-only.
15. **Neck threshold scope**, **TIES weighting**, **surgery detection**, **MoE in Champ**, **G2/G3**: as in version 2.

9 Changelog

What the first draft of version 3 got wrong

The first draft of this version (also dated 2 November) was written from a 25 October copy of `hodgemix-go`, not from the build machine's current tree. Against the 2 November pull it was wrong in these places:

- **Towers.** It described `copyTowers`, `assimilateGemmaTowers` and `writeBridge` (TOWERS T1–T4, `bnzmix.embed_bridge`, Gemma 4 encoders carried or ternarised). All are removed; `assimtowers.go` is deleted. Input towers are not written, and the only towers are the L-A ones.
- **Seed.** It described a Re-Basin/Hungarian assignment. The code is gap selection (`train.Align`).
- **Lock.** It described a Kuramoto/KTE population anneal with early stop in `synthD` and `-lock v2`. Both are now a window certificate, and the anneal survives only in `lockLimb`.
- **No-lock fallback.** It said an unlocked tensor keeps the base grid. That fallback is removed: the curl residual is kept and takes the Ricci–DeTurck step.
- **Ricci–DeTurck step, `train/` package, `vocab.float.safetensors`, served run file, `promoteRope`, wedge gate.** All were absent from it.
- **Repair.** It said "≤ 3 passes" with `-repair-passes`. The flag is removed and the loop is uncapped.
- **Preflight bug.** It reported preflight failing unstamped bases on the quantization names. That check is gone, and the bug is fixed.
- **Tests.** It reported 74 pass, 8 skip and 1 fail. The current tree gives 52/8/1, with two married-topology tests exiting the binary.

- **Shapes, BNZ4 embedding chunk, run files.** It called the 2B4T shapes hard-coded (they come from config), gave a fixed $8192 \cdot 2560 \cdot 2$-byte chunk (now 8192 rows of the tensor's own width), and cited `runs/hodge-4.yaml` rather than the `-instruct` run files.

Changelog from version 2 (code of 2 November)

- **Runtime target hkruntime**: `stampHodge` renames to `hodge` and `HodgeForCausalLM` and maps the quantization names. It runs after REPAIR, on the run file's output config.
- **Champ/BNZ4 legacy**, still executed. `champRuntime` no longer checks the quantization names and accepts `mrope`. The BNZ4 embedding chunk is 8192 rows of the tensor's own width.
- **Gemma 4**: `languageModel`, `embedKey`, and `asideTensor` kept out of the mix *and the output*; per-layer `FBy`/`KVBy`/`QBy`.
- **Towers**: L-A vision and audio towers and least-squares projectors on the base embedding (`writeTowers`) on every run. No input tower, encoder or bridge is written.
- **Seed**: phase-correlation shift + gap selection (`placeSource`, `train.Align`). Re-Basin/Hungarian is deleted.
- **Lock**: window certificate (Alg. D step 5, and `gapLock` for `-lock v2`); no KTE anneal in either.
- **One Ricci–DeTurck step** (`train.Flow`) on every synthesized tensor, with a lattice-neck cap to the base.
- **No band / no lock**: the curl residual is kept for the step; no base-grid fallback.
- **splineGrid**: smoothed commuting 1-form (`fec.md` §6).
- **Limbs**: body and neck stops at 2^{-8}; wide neck $> 1/5$; the limb lock keeps the anneal early stop.
- **Repair**: uncapped passes to spread < 0.2; `-repair-passes` is removed.
- **Run file**: `config` is required everywhere; `OUT.yaml` is written twice, the second time with the served models.
- **Outputs**: `volume.safetensors` and the new `vocab.float.safetensors`.
- **Reference forward**: tower slots at $|V|$ and $|V| + 196$, RoPE phase on planes 56–63, `checkTower`. New: the *wedge gate* on both residual adds, chat formats that follow the tokenizer, Qwen-VL families, `rope_parameters`.
- **Outside the pipeline**: `itt/`, `turn/`, `towerwasm/`, `eval/chat.go`, `eval/converse.go`, cochain digons, and the `stdiff` content match.

Changelog from version 1 (kept from version 2)

- TIES: λ-scaled agreeing mean replaced by agreeing sum over total weight (`ternaryTIESAll`).
- Limbs: T1 neck cut, Kuramoto lock of body limbs to limb 0, 4 neck limbs every run, llama3 body pack (6 limbs, tol 0.02), `neck.go`; Champ limb checks.
- Repair: base-stored gains only; inserted sub-norms fixed at 1; `pre_feedforward_layernorm`; stop at spread < 0.2 (now uncapped).
- Family-general reference forward and kernels (`ACT`, `GEMMA`, `SCALE`, `DENOM`, `SCALING`); sliding-window and MoE configs refused by Champ.
- gpt-oss MoE forward with MXFP4 gather (G1); update-expert admission (G2); page engine gate, temperature mixture and presence penalty (G3).
- Volumes, flow currents and cochain Hodge split on CPU and GPU.
- Run file, preflight, fail-stop policy; per-model chat formats (`chatfmt`); tools `spm2json`, `stdiff`.
- Fixed: the reference forward no longer applies W_k twice.

Acknowledgment

The surgery cap (§3.13) rests on Grigori Perelman's Ricci flow with surgery on three-manifolds [26].

A Claim index (file: function)

Claim	Source
Shapes from config, calibration texts	`eval/arch.go`, `eval/model.go`; `eval/eval.go: Texts, Prompts, calibration`
Wedge gate	`eval/wedge.go: wedgeKeep, wedgeArea, admitWrite`; `eval/eval.go: feed`
Chat formats	`eval/chat.go`
Per-token int8 absmax	`eval/eval.go: quant`
Block equations, relu^2, sub-norms, GQA	`eval/eval.go: feed`
RoPE pairing	`eval/eval.go: rope`
Tied head, LM head skipped	`eval/eval.go: logits, step, feed`
Massive channels, spike units, means	`eval/eval.go: FindSingular`
LUT block size, tables, gather	`eval/lut.go: blockFor, newLimb, input.tables, lin.applyIn`
LUT = MAC	`eval/lut_test.go: TestLUTMatchesMAC, TestMultiLimb`
WGSL CENTER, ACCUM; Champ limb dispatch (not in tree, v2)	`bitnet/kernels/bitlinear.wgsl`; `bitnet/model/model.go`
Union vocab, embedding map	`vocab.go: buildUnion, residualMap, cholSolve`
QK-norm fold, RoPE remap	`attention.go: prepQK, ropeRemap`
Sub-norms added, q/k_norm and tied head dropped	`attention.go: champLayout`
Seed	`math.go: placeSource, seedShift, seedAssign`; `train/select.go: Align, match, GapNeed`
Window lock	`dtft.go: synthD` (step 5); `synth.go: gapLock`
Ricci–DeTurck step	`synth.go: flowGrid`; `train/ricci.go: Flow, StepResidual, Lap2, DT`; `train/hardy.go: Rho, CH, Omega`
Base scale, ternarize, LS scale, limbs call	`main.go` (synthesis loop)
Pin-mean	`pinmean.go: meanInput, pinMean`
Limbs	`limbs.go: quantLimb, residualLimbs, writeLimbs, zeroLimbs, refillLimbs`
OV chart	`ovchart.go: solveSPD, ternarize, ovChart`
Surgery	`surgery.go: baseCodes, surgeryWith`
Stage order	`main.go` (calibrate, union, synthesis, OV chart, surgery, repair, stage 5)
`-roles` subset copies scale; NaN in report	`main.go` (unmixed packed copy; JSON non-finite to string)
Config from tensors	`topology.go: configFor`; `main.go`
Champ checks C1–C10	`champ.go`
Embedding segments	`bnz4/container.go`
Families, config parsing	`eval/arch.go`; `eval/model.go`
gpt-oss mixture, top-k, SwiGLU	`eval/eval.go: mixture, topkSoftmax, swiglu, mixLin`
MXFP4 gather	`eval/mxfp4.go: mx4.gather, SplitGateUp`
Update-expert gate	`eval/eval.go: attach, consider, denseExpert, admit, hotCell`; `eval/files.go: SetUpdate`
Engine gate (not in tree, v2)	`bitnet/web/router.js`; `bitnet/main.go`

Claim	Source
TIES	`math.go: ternaryTIESAll`; `ties_test.go`
Neck, limbs, lock	`limbs.go: t1Tau, singularNeck, residualLimbs, lockLimb, neckLimbs`; `neck.go: packNeck, appendSing`
Repair	`repair.go: normTensor, repairRatio, baseOwnsGain, repair, repairStop`
Run file, preflight	`runfile.go: loadRunFile, writeRunFile`; `preflight.go: preflightCheck`; `main.go: bindServed`
Champ refusals, limb checks, RoPE	`champ.go: champRuntime, champLimbs, promoteRope`; `eval/arch.go: RopeDict, RopeType`
Kernel overrides (not in tree, v2)	`norm.wgsl, embed.wgsl, attention.wgsl, rope.wgsl`
Volumes, vocab float	`volume.go: volumes, moeAxes, writeExpertVolumes`; `vocabfloat.go: vocabFloat, vocabCochain, cochainFloat`
Hodge names	`hodgename.go: stampHodge, stampHodgeFile, ternaryFamily`; call after REPAIR in `main.go`
Gemma 4 inputs	`main.go: languageModel, embedKey, towerTensor, asideTensor`; `topology.go: FBy, KVBy, QBy, fAt, finishF`
TOWERS	`main.go` (aside count, `not_written`); `towers.go: writeTowers, probeCount, sampleRows, fitProjector, writeEmbed, writeTowerConst`; `tower/`
Tower slots, RoPE phase, checks	`eval/tower.go: install, Vision, checkTower`; `eval/eval.go` (planes 56–63)
Smoothed 1-form	`math.go: splineGrid, newAxisGeom, factorMass`; `fec.md` §6
Anneal early stop (limb lock only)	`kuramoto.go: runAnnealStage, annealStall, kuramotoTick`; `limbs.go: lockLimb`
No band / no lock	`dtft.go: synthD`
Limb stops, wide neck	`limbs.go: residualLimbs, neckWide, neckLimbs`
Champ check (legacy)	`champ.go: champCheck, champRuntime`; `main.go` (`of.Config`)
Currents, cochain, digons	`eval/flow.go`; `cochain.go: newFlowComplex, hodge, harmonic, pairFaces, flowRun`
Tools	`itt/`, `turn/turn.go: Circuit, Child`, `towerwasm/`, `stdiff/`

References

[1] G. Disney-Leugers, Hodge–Kuramoto ternary model merging (paper_v2), Encryptid Systems, 2026.

[2] G. Disney-Leugers, DTFT selective lock (dtft_lock), Encryptid Systems, 2026.

[3] G. Disney-Leugers, Singularity surgery for Hodge–Kuramoto model merging, Encryptid Systems, 2026.

[4] S. Ma et al., The era of 1-bit LLMs: all large language models are in 1.58 bits, arXiv:2402.17764, 2024.

[5] S. Ma et al., BitNet b1.58 2B4T technical report, arXiv:2504.12285, 2025.

[6] J. Wang et al., 1-bit AI infra: Part 1.1, fast and lossless BitNet b1.58 inference on CPUs, arXiv:2410.16144, 2024.

[7] J. Wei et al., T-MAC: CPU renaissance via table lookup for low-bit LLM deployment on edge, arXiv:2407.00088, 2024.

[8] P. Yadav et al., TIES-Merging: resolving interference when merging models, NeurIPS 2023, arXiv:2306.01708.

[9] S. K. Ainsworth, J. Hayase, S. Srinivasa, Git Re-Basin: merging models modulo permutation symmetries, ICLR 2023, arXiv:2209.04836.

[10] K. Jordan et al., REPAIR: renormalizing permuted activations for interpolation repair, ICLR 2023, arXiv:2211.08403.

[11] M. Sun et al., Massive activations in large language models, arXiv:2402.17762, 2024.

[12] G. Disney-Leugers, Multi-Radix Geometry, Encryptid Systems, 2026.

[13] G. Disney-Leugers, A bounded commuting projection for the married-axis grid (`fec.md`), Encryptid Systems, 2026.

[14] G. Disney-Leugers, Monodromy, the Riemann–Hilbert problem and mixture-of-experts on the hodgemix weight 3-manifold, Encryptid Systems, 2026.

[15] OpenAI, gpt-oss-120b & gpt-oss-20b model card, arXiv:2508.10925, 2025.

[16] W. Fedus, B. Zoph, N. Shazeer, Switch Transformers: scaling to trillion parameter models with simple and efficient sparsity, arXiv:2101.03961, 2021.

[17] D. Lepikhin et al., GShard: scaling giant models with conditional computation and automatic sharding, arXiv:2006.16668, 2020.

[18] B. D. Rouhani et al., Microscaling data formats for deep learning, arXiv:2310.10537, 2023.

[19] M. Yu et al., The super weight in large language models, arXiv:2411.07191, 2024.

[20] J. Su et al., RoFormer: enhanced transformer with rotary position embedding, arXiv:2104.09864, 2021.

[21] B. Zhang, R. Sennrich, Root mean square layer normalization, NeurIPS 2019, arXiv:1910.07467.

[22] J. Ainslie et al., GQA: training generalized multi-query transformer models from multi-head checkpoints, arXiv:2305.13245, 2023.

[23] D. R. So et al., Primer: searching for efficient transformers for language modeling, arXiv:2109.08668, 2021.

[24] H. Wang et al., Foundation transformers, arXiv:2210.06423, 2022.

[25] G. Perelman, The entropy formula for the Ricci flow and its geometric applications, arXiv:math/0211159, 2002.

[26] G. Perelman, Ricci flow with surgery on three-manifolds, arXiv:math/0303109, 2003.

[27] G. Perelman, Finite extinction time for the solutions to the Ricci flow on certain three-manifolds, arXiv:math/0307245, 2003.